

Extractor de prototipus de comportament o perfils

Identificador de l'equip: 13.3

Gil Martin, Samuel: samuel.gil@estudiantat.upc.edu

Leivar Guiu, Artur: artur.leivar@estudiantat.upc.edu

Mestre Tugores, Tomeu: tomeu.mestre@estudiantat.upc.edu

Turro Gomez, Pau: pau.turro.gomez@estudiantat.upc.edu

Forcadell Moral, Joel: joel.forcadell@estudiantat.upc.edu

Versió de lliurament 2.1

2^a entrega PROP

17/11/2025

Índex

1. Diagrama de model conceptual.....	5
1.1 Disseny del diagrama de model conceptual.....	5
2. Diagrama de classes capa presentació.....	7
2.1 Disseny del diagrama de classes capa presentació.....	7
2.2 Breu descripció de les classes de presentació.....	8
2.2.1 LoginView.....	8
2.2.2 SignUpView.....	8
2.2.3 MainMenuView.....	8
2.2.4 VistaContestar.....	9
2.2.5 VistaCrearEditarForm.....	9
2.2.6 VistaAnalyze.....	10
2.2.7 VistaResultAnalyze.....	10
3. Diagrama de classes capa persistència.....	11
3.1 Disseny del diagrama de classes capa persistència.....	11
3.2 Breu descripció de les classes de persistència.....	12
3.2.1 CtrlPersistence.....	12
3.2.2 CtrlUser.....	12
3.2.3 CtrlFormQuestion.....	12
3.2.4 CtrlAnalyze.....	12
3.2.5 CtrlAnswerQuestion.....	12
3.2.6 Cache.....	13
3.2.7 GsonFactory.....	13
3.2.8 FileManager.....	13
3.2.9 DataPath.....	13
4. Estructura de dades i algorismes utilitzats.....	14
4.1 Algorismes.....	14
4.1.1 K-Means.....	14
4.1.2 K-Medoids.....	16
4.2 Estructures de dades.....	17
4.2.1 Answer.....	17
4.2.2 AnswerMultipleChoiceQuestion.....	18
4.2.3 MultipleChoiceQuestion.....	18
4.2.4 AnalysisResult.....	18
4.2.5 Analyze.....	19
4.2.6 AnalyzeStrategy.....	20
4.2.7 KMeansStrategy.....	21
4.2.8 KMedoidsStrategy.....	22
4.2.9 Persistència.....	22
5 Millors en algorismes i estructures de dades.....	24
5.1 Algorismes.....	24
5.1.1 K-Medoids.....	24
5.2 Estructures de dades.....	25

Restriccions Textuals

1. Claus externes: (User, idUser), (Form, idForm), (Answer, User+Form), (Question, Form+idQuestion), (AnswerToQuestion, Answer+Question), (Analyse, Form+time)
2. Les preguntes respostes a un "Answer" han de pertànyer al "Form" de l'"Answer".
3. La classe "AnswerToQuestion" és "NumericalAnswer" si la "Question" relacionada és "NumericalQuestion".
4. La classe "AnswerToQuestion" és "FreeAnswer" si la "Question" relacionada és "FreeQuestion".
5. La classe "AnswerToQuestion" és "MultipleChoiceAnswer" si la "Question" relacionada és "MultipleChoiceQuestion".
6. La classe "AnswerToQuestion" és "SingleChoiceAnswer" si la "Question" relacionada és "SingleChoiceQuestion".

Totes les descripcions de les classes i les explicacions dels seus mètodes es troben detallats al Javadoc, que hem actualitzat per a la segona entrega amb els mètodes que s'han afegit.

2.2 Breu descripció de les classes de presentació

2.2.1 LoginView

Propòsit:

Finestra d'inici de sessió per a l'aplicació Form Builder.

Descripció:

Aquesta classe representa la interfície gràfica on els usuaris poden accedir a l'aplicació de dues formes:

1. Usuaris registrats: Introduint el seu ID d'usuari i contrasenya
2. Usuaris convidats: Accedint de manera anònima amb funcionalitat limitada

Funcionalitats:

- Validació de credencials d'usuaris registrats
- Creació d'usuaris temporals (convidats)
- Redirecció al registre de nous usuaris
- Navegació al menú principal després d'autenticació reeixida

2.2.2 SignUpView

Propòsit:

Finestra de registre de nous usuaris per a l'aplicació Form Builder.

Descripció:

Aquesta classe representa la interfície gràfica on els usuaris poden crear un nou compte en el sistema.

Funcionalitats:

- Registre de nous usuaris amb email, nom d'usuari i contrasenya
- Validació de dades d'entrada (camp obligatoris, format d'email)
- Confirmació de contrasenya per a evitar errors d'escriptura
- Redirecció a la finestra de login després de registre reeixit
- Navegació a la finestra de login per a usuaris existents

2.2.3 MainMenuView

Propòsit:

Menú principal de l'aplicació Form Builder després de l'inici de sessió.

Descripció:

Aquesta classe representa la interfície principal de l'aplicació on els usuaris poden gestionar els seus formularis i accedir a totes les funcionalitats.

Funcionalitats:

- Dashboard amb estadístiques de formularis (totals, anàlisis, respostes)
- Gestió completa de formularis (crear, veure, editar, eliminar)
- Navegació per diferents seccions (dashboard, els meus formularis, configuració, ajuda)
- Diferenciació entre usuaris registrats i convidats (funcionalitat limitada)
- Visualització de formularis de l'usuari amb opcions d'edició, resposta i anàlisi
- Sistema de navegació amb menú lateral i barra superior
- Tancament de sessió amb confirmació

2.2.4 VistaContestar

Propòsit:

Finestra per a la creació i edició de formularis en l'aplicació Form Builder.

Descripció:

Aquesta classe representa la interfície gràfica principal per a la gestió completa de formularis. Permet als usuaris crear nous formularis des de zero o editar formularis existents amb una interfície intuïtiva i visualment atractiva.

Funcionalitats:

- Mode creació: construir nous formularis des de zero
- Mode edició: modificar formularis existents
- Afegir preguntes de diferents tipus (text lliure, opció múltiple, numèriques i opcions ordenades)
- Gestió de preguntes existents amb visualització del seu estat (modificables o només lectura)
- Validació automàtica quan un formulari ja té respostes (mode només lectura)
- Interfície arrastrar i deixar per a reorganitzar preguntes
- Previsualització en temps real del disseny del formulari
- Indicadors visuals de progrés i nombre de preguntes
- Confirmació de sortida amb canvis no guardats

2.2.5 VistaCrearEditarForm

Propòsit:

Finestra per a respondre formularis en l'aplicació Form Builder.

Descripció:

Aquesta classe representa la interfície gràfica completa per a la resposta de formularis. Ofereix una experiència d'usuària optimitzada amb navegació fluïda entre preguntes i gestió intel·ligent de respostes temporals.

Funcionalitats:

- Càrrega dinàmica de formularis reals des del controlador de presentació
- Adaptació automàtica a diferents tipus de pregunta (text, numèric, opció múltiple, opció única)
- Navegació seqüencial entre preguntes amb indicadors de progrés

- Guardat automàtic de respostes temporals en memòria
- Validació en temps real de respostes (rangs numèrics, longitud màxima)
- Mode complet de pantalla per a millor experiència d'usuari
- Opció de guardar progrés i sortir
- Confirmació d'enviament final de respostes
- Gestió de preguntes opcionals
- Integració amb el sistema de persistència per a guardat definitiu

2.2.6 VistaAnalyze

Aquesta vista s'encarrega de mostrar tots els anàlisis fets anteriorment sobre el formulari seleccionat. D'aquests en mostra una informació reduïda (quan es va fer, el nombre de clústers, l'algorisme utilitzat i el coeficient de qualitat) i dona la possibilitat de seleccionar-ne un per a obtenir informació més detallada.

A més a més permet crear un nou anàlisis del formulari.

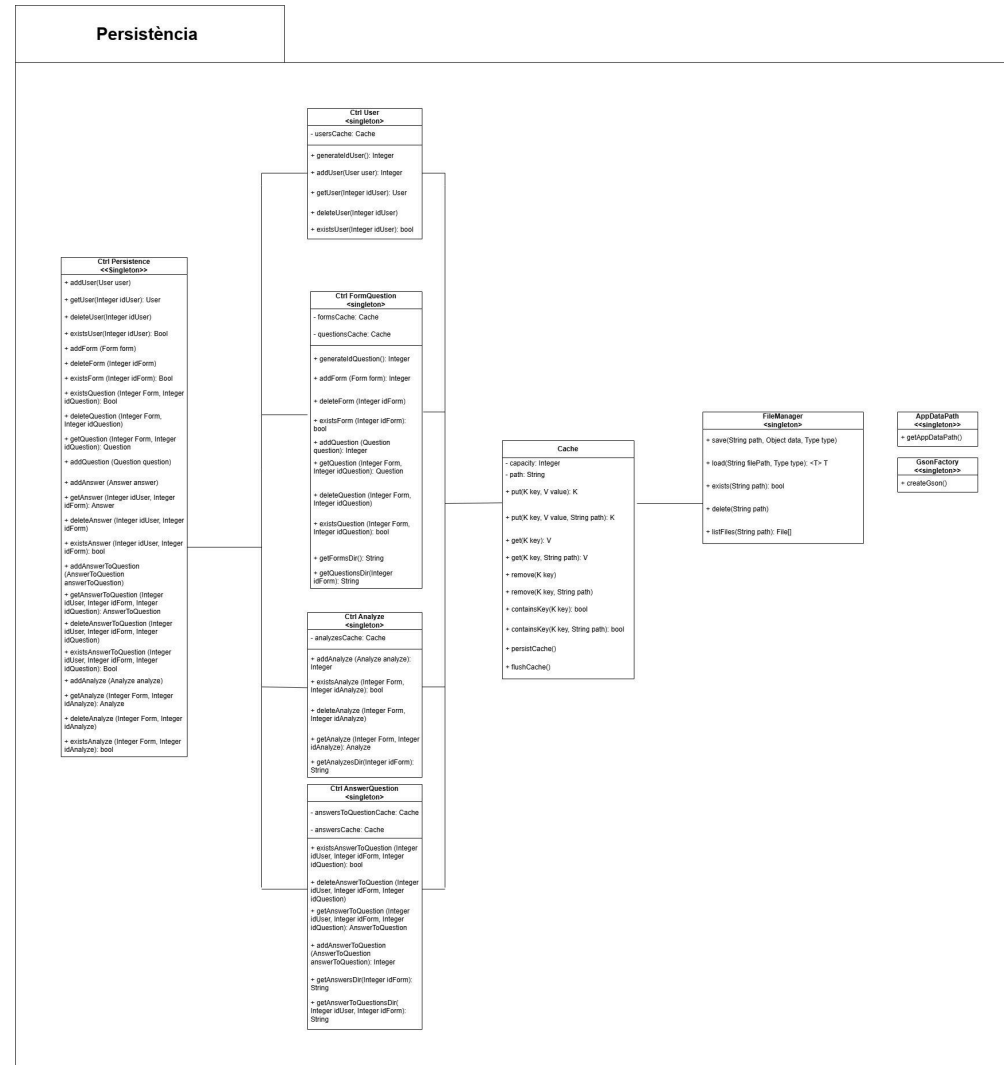
2.2.7 VistaResultAnalyze

Aquesta vista mostra informació detallada sobre una clusterització. Aquesta consisteix en la informació que ja donava VistaAnalyze però ara a més a més tenim una representació gràfica de les respostes i de la seva clusterització.

3. Diagrama de classes capa persistència

3.1 Disseny del diagrama de classes capa persistència

Aquí mostrem el nostre diagrama UML de persistència. El diagrama es troba detallat al directori DOCS



3.2 Breu descripció de les classes de persistència

3.2.1 CtrlPersistence

Controlador genèric de la capa de persistència, aquest s'encarrega de distribuir les diferents operacions de persistència en els controladors específics. Hem separat el controlador en les següents classes: `ctrlUser.java`, `ctrlFormQuestion.java`, `CtrlAnalyze.java` i `CtrlAnswerQuestion.java`

3.2.2 CtrlUser

Controlador de persistència específic de User. S'encarrega de totes les operacions relacionades amb la persistència de la classe User. Emmagatzema Users en disc i memòria usant la Cache. Assigna IDs de manera automàtica si no són proporcionats. La classe segueix el patró de singleton.

3.2.3 CtrlFormQuestion

Controlador de persistència específic de Form i Question. S'encarrega de totes les operacions relacionades amb la persistència de les classes Form i Question. Emmagatzema els objectes en disc i memòria usant la Cache. Assigna IDs de manera automàtica si no són proporcionats. La classe segueix el patró de singleton.

3.2.4 CtrlAnalyze

Controlador de persistència específic de Analyze. S'encarrega de totes les operacions relacionades amb la persistència de la classe Analyze. Emmagatzema Analyzes en disc i memòria usant la Cache. Assigna IDs de manera automàtica si no són proporcionats. La classe segueix el patró de singleton.

3.2.5 CtrlAnswerQuestion

Controlador de persistència específic de Answer i AnswerToQuestion. S'encarrega de totes les operacions relacionades amb la persistència de les classes Answer i AnswerToQuestion. Emmagatzema Analyzes en disc i memòria usant la Cache. Assigna IDs de manera automàtica si no són proporcionats. La classe segueix el patró de singleton.

3.2.6 Cache

Implementació d'una memòria cache senzilla 'WriteAllocate' amb operacions put, get, delete i exists.

S'implementa com a variant d'una memòria cache "CopyBack", amb la diferència que sempre escriu un objecte al disc quan aquest s'ha de substituir, ja que no sap si l'objecte ha estat modificat.

Implementa persistCache(), que s'ha de cridar ****abans**** que una instància d'aquesta classe sigui destruïda.

La memòria cau:

- Escriu al disc quan s'ha de substituir un objecte.
- Escriu al disc quan s'elimina un objecte.
- Escriu al disc com a màxim "capacitat" vegades quan es crida a persistCache().
- Carrega del disc quan s'ha d'obtenir o comprovar l'existència d'un objecte, i només quan no es troba aquest prèviament a la memòria cache.

3.2.7 GsonFactory

GsonFactory simplement té un mètode de getGson, que retorna un objecte Gson personalitzat. La configuració per defecte de Gson no ens serveix perquè hem de serialitzar i deserialitzar a/de json algunes classes jeràrquiques, i estem usant la llibreria de Gson-extras per afegir un camp "type" als arxius i que Gson detecti de manera automàtica a quina classe específica pertany cada json.

3.2.8 FileManager

Aquesta classe implementa un patró de disseny singleton. La classe FileManager utilitza la llibreria Gson per a la serialització i escritura en disc, i per a la lectura i deserialització d'objectes java. Utilitzant el format de json.

3.2.9 DataPath

Aquesta classe només està pensada per cridar a getAppDataPath(). Recupera i torna el camí correcte on desar les dades, depenent del sistema operatiu que s'utilitzi (Windows, Unix, BSD o una variant semblant a Unix com Linux)

4. Estructura de dades i algorismes utilitzats

4.1 Algorismes

4.1.1 K-Means

Algorisme:

L'algorisme de K-means és un algorisme de clustering que funciona com es descriu a continuació:

1r Inicialitzar k centroides.

2n Assignar cada resposta existent al centroide més proper, creant els clusters.

3r Recalculer els centroides per a cada cluster.

4t Repetir 2n i 3r pas fins que ja no hi hagi cap canvi als centroides o s'hagi assolit cert llindar d'iteracions.

En el nostre cas sempre farem una quantitat fixa d'iteracions perquè és més simple d'implementar. Queda com a objectiu per a la propera entrega millorar-ho.

L'algorisme k-means no assegura un òptim global però sí un òptim local donats els centroides inicials optimitzant el quadrat de les distàncies. És a dir, que la distància entre cada punt al centroide del seu cluster al quadrat és mínima.

La nostra implementació:

Primer de tot inicialitzem els centroides amb k-means++ (explicat més endavant).

Aquesta inicialització la fem amb els índexs que ocupen les respostes triades a l'ArrayList d'entrada per a no haver de copiar aquest darrer múltiples cops.

Un cop tenim els centroides inicials triats repetim el següent fins a trobar el resultat final (que en el nostre cas tan sols ens aturem quan hem assolit una certa quantitat d'iteracions per a no haver de crear còpies extres dels centroides i perquè calcular si són iguals seria costós):

- Donats els centroides que tenim calculem per cada resposta quin centroide té més aprop i l'assignem al seu cluster. Això ho fem mirant a quina distància es troba de cada centroide.
- Un cop generada la clusterització recalculem el centroide de cada cluster.

Per a recalculer cada centroide seguim el següent criteri (noti's que el centroide no cal que sigui una resposta ja existent):

- Si tenim una resposta a pregunta que és numèrica, el centroide tindrà com a resposta a aquesta pregunta la mitjana dels valors que té aquesta resposta dins del clúster.
- Si tenim una resposta a pregunta que és de resposta única el valor que pren és la moda de les altres respostes.
- Si tenim una resposta a pregunta que és de selecció múltiple o lliure triem d'entre totes les respostes donades quina és la que es troba a menys distància de les altres.

Finalment, per a adaptar-nos al format de sortida hem de dir a quin cluster pertany cada resposta.

Cost temporal:

$O(nc * ni * na * np)$ on nc és el nombre de clusters, ni el nombre d'iteracions que fa el programa fins acabar (en el nostre cas fixe a 100), na el nombre de respostes sobre les que hem de crear els clusters i np el nombre de preguntes que té el formulari.

El cost és aquest perquè per cada iteració (ni) hem de calcular a quin cluster va associada cada resposta (na). I per cada resposta hem de mirar a quina distància es troba de cada centroides (nc) mirant la distància local per cada pregunta (np).

Cost espacial:

$O(na + nc)$

El cost és aquest perquè necessitem espai extra per a:

Guardar-nos la divisió que hem fet de les respostes (na)

Guardar-nos els centroides (nc)

Guardar-nos a quin cluster està assignada cada resposta (na)

Llavors ens quedaria cost $2na + nc$, però com que usem notació asimptòtica ens queda $na + nc$.

K-means++

Degut a la dependència que té de l'elecció inicial de centroides és crucial el inicialitzar-los de forma que el resultat sigui el més proper a l'òptim possible. Per a això existeix l'algorisme k-means++. Aquest algorisme funciona de la següent manera:

1r Tria un centroide inicial aleatòriament

2n Per cada resposta li assigna una possibilitat de ser escollit com a centroide que és igual a:

$$\frac{D(x')^2}{\sum_{x \in \mathcal{X}} D(x)^2}$$

On $D(x)$ representa la distància mínima entre la resposta x i tots els centroides i $\mathcal{X}$ és el conjunt de totes les respostes al formulari.

3r Triem el nou centroide. Noti's com les respostes que ja són centroides no podran ser triades de nou perquè la seva $D(x)$ corresponent serà 0.

Implementació:

Per a implementar K-means++ requerim de 3 funcions auxiliars: una per a calcular $D(x)$, una altra per a calcular el sumatori de totes aquestes distàncies i una altra per a donar un nombre aleatori seguint una distribució de probabilitats.

A més a més, com que tots els centroides són respostes existents per a estalviar memòria retornem un `ArrayList<Integer>` on cada element representa l'índex del centroide dins de la matriu de respostes. Això representa una millora davant de retornar els valors dels centroides amb un `ArrayList<ArrayList<AnswerToQuestion>>` perquè un `Integer` ocupa

menys que un `ArrayList<AnswerToQuestion>`. En temps, però, no hi hauria una diferència significativa.

Cost temporal:

Siguin nc i na el nombre de clusters que volem generar (i per tant el nombre de centroides) i el nombre de respostes que tenim, respectivament. Llavors el cost de `k-means++` és de $O(nc * na * nc)$ perquè el bucle exterior (i) fa nc iteracions del bucle interior (j), el qual fa na iteracions. I cada iteració del bucle intern té cost igual al nombre de centroides que ja s'han assignat i per tant és proporcional a nc .

Es pot veure si es resol la recurrència següent:
$$\sum_{i=1}^{nc} \sum_{j=1}^{na} \sum_{k=1}^i k = \sum_{i=1}^{nc} na * i = na * \sum_{i=1}^{nc} i$$

que, en notació asimptòtica, és $na * nc^2$.

Cost espacial:

$O(nc)$, ja que només hem de crear el vector de centroides i la matriu de respostes es passa per referència i per tant no es copia.

4.1.2 K-Medoids

L'algorisme K-Medoids que s'ha implementat és un PAM, *partition around medoids*, de la mateixa manera que `k-means`, `k-medoids` és un algorisme de clustering.

La base de l'algorisme és la següent:

- Primer s'inicialitzen k "medoids" de manera aleatòria (aquests medoids són punts reals, no com en el cas de `K-means`, en el nostre cas seleccionam k respostes dels usuaris)
- S'assigna cada punt (respostes de usuari) a cada medoid més proper
- Mentre el cost global del clustering segueixi disminuint, iteram de la següent manera
 - Per cada medoid, feim un swap amb un punt no medoid
 - Si el swap empitjora el cost global, el descartam

La nostra implementació funciona de la següent manera:

Primer de tot precalculam una matriu amb totes les distàncies de les respostes, això és especialment útil per reduir el cost quan les respostes tenen un cert tamany i hi ha una gran quantitat de respostes, ja que la complexitat de l'algorisme és bastant elevada. Encara que sacrificam memòria, no tocaria ser un problema en cap cas, doncs estem guardant una matriu de $n*n$ doubles.

Cridam a la funció `initializeMedoids()`, que inicialitza els k medoides de manera aleatòria. Llavors, senzillament iteram en un bucle fins que arribam a un màxim d'iteracions predefinit `maxIterations`, o l'algorisme ha convergit i ja no obtenim un cost global menor que l'actual (no hi ha cap swap vàlid).

En aquest bucle iteram de la següent manera:

Mentre la funció `swap` no retorni `-1`, o no haguem arribat al màxim nombre d'iteracions, per a cada medoid, cridam a la funció `swap`.

S'ha usat aquesta funció auxiliar `findBestSwap` que sempre retorna el millor swap possible per a un medoid i , d'aquesta manera, en cada iteració asseguram que feim un canvi de medoid i no-medoid que beneficia el cost total, i sinó, podem assegurar que aquest no existeix i retornem un `-1`. En cas de que hi hagi un swap correcte, és retornem l'índex del punt amb el que s'ha de fer el swap.

Una implementació més eficient hagués estat fer el càlcul dels costos de manera incremental, ja que en el nostre cas, en la funció `swap`, simulem el swap, reconfigurem l'estat dels clusters, calculam la distància total, i amb aquesta distància sabem quin és el swap millor, i feim aquest procés en cada iteració, això afegeix una complexitat temporal molt gran.

Queda com a tasca per a següents entregues implementar aquestes millores d'eficiència.

Per acabar, quan l'algorisme ha convergit en la millor configuració possible o hem arribat al nombre màxim de iteracions, calculam la mètrica de qualitat amb Silhouette, i posteriorment, construim un objecte `AnalyseResults` per poder retornar el resultat de l'algorisme a la classe `Analyze`.

Sigui n la quantitat de respostes, k el nombre de clusters, T el nombre d'iteracions del bucle. La complexitat en el càlcul de distàncies és $O(n^2)$, assignació de clusters $O(n*k)$ i per al bucle de swaps $O(n^2*k)$. Per tant, el cost de l'algorisme és $O(n^2*k^2*T)$, podem obviar T perquè sabem que com a màxim serà `maxIter`, que és constant

4.2 Estructures de dades

4.2.1 Answer

Tal i com hem especificat anteriorment, la classe `Answer` representa la submissió d'un usuari a un formulari concret. Aquesta classe guarda l'identificador del formulari, l'identificador de l'usuari, el moment en què s'ha creat la resposta i, finalment, la llista d'identificadors de les preguntes contestades. Per analitzar les estructures de dades possibles, definim la variable x com el nombre de preguntes contestades per l'usuari, i la variable n com el nombre total de preguntes del formulari. D'aquesta manera, podem dir que el límit de respostes és $0 \leq x \leq n$.

A l'hora de decidir com implementar l'agrupació de respostes, vam considerar diverses opcions:

Array d'enters (`Integer[]`) L'estructura més senzilla per representar un conjunt de respostes és un array.

Avantatges: accés per índex en temps constant $O(1)$.

Inconvenients: mida fixa, insercions i eliminacions costoses $O(n)$. No és pràctic quan el nombre de respostes varia dinàmicament.

`ArrayList<Integer>` Implementa una llista dinàmica basada en un array intern.

Avantatges: creix automàticament, accés per índex ràpid $\Theta(1)$, mètodes ja implementats (add, remove, contains).

Inconvenients: insercions i eliminacions al mig de la llista tenen cost $\Theta(n)$. Tot i això, en el nostre cas les operacions són principalment afegir o eliminar respostes concretes, i el cost és acceptable.

LinkedList<Integer> Llista doblement enllaçada.

Avantatges: insercions i eliminacions eficients $\Theta(1)$ si es té la referència al node.

Inconvenients: accés per índex lent $\Theta(n)$, més consum de memòria. No és necessari en aquest context, ja que les consultes per índex són freqüents.

HashSet<Integer> Conjunt que evita duplicats i permet cerca ràpida.

Avantatges: operacions de cerca, inserció i eliminació en temps mitjà $\Theta(1)$.

Inconvenients: no manté ordre ni permet accés per índex. En el nostre cas, volem conservar l'ordre de les respostes i poder consultar-les seqüencialment.

Finalment, es va escollir ArrayList<Integer> per implementar l'atribut responses de la classe Answer. Aquesta estructura ofereix un bon equilibri entre simplicitat, flexibilitat i eficiència per a les operacions més habituals: afegir respostes, eliminar-les i comprovar si una pregunta ha estat contestada. Tot i que altres opcions com HashSet podrien millorar la cerca o evitar duplicats, en aquest context es prioritza mantenir l'ordre i la facilitat d'ús.

4.2.2 AnswerMultipleChoiceQuestion

Estructura: List<Integer>

Raons de l'elecció:

- Flexibilitat en la implementació: Utilitzar List permet canviar la implementació subjacent (ArrayList, LinkedList) sense alterar la interfície
- Abstracció adequada: L'usuari de la classe només necessita saber que és una llista, no com està implementada
- Operacions de col·lecció estàndard: Accés a tots els mètodes de la interfície List (add, remove, contains, etc.)
- Manteniment de l'ordre: Conserva l'ordre de selecció de les opcions

4.2.3 MultipleChoiceQuestion

Estructura: ArrayList<String>

Raons de l'elecció:

- Accés aleatori òptim ($\Theta(1)$): Les opcions es consulten freqüentment per índex
- Iteració eficient: Millor localitat de memòria cau per recórrer totes les opcions
- Mida previsible: El nombre d'opcions és estable i moderat
- Operacions adequades: Afegir opcions és eficient i eliminar, encara que $\Theta(n)$, és acceptable

Avantatge del disseny consistent: Ambdues classes utilitzen ArrayList, creant coherència en el codi i facilitant el manteniment del sistema de formularis.

4.2.4 AnalysisResult

La classe AnalysisResult guarda tres elements principals:

- `int[] clusterAssignments`: un array on la posició *i* indica el clúster assignat a l'usuari.
- `ArrayList<ArrayList<AnswerToQuestion>>` `centroids`: una matriu dinàmica que representa els centroides o medoides de cada clúster.
- `double quality`: un valor numèric que mesura la qualitat de l'agrupació.

Per analitzar les estructures de dades possibles, definim les variables següents:

n: nombre d'usuaris.

k: nombre de clústers.

m: nombre de preguntes que defineixen cada centroid.

D'aquesta manera, podem dir que la mida de l'array `clusterAssignments` és *n*, mentre que la matriu de centroides té dimensió $k \times m$.

Opcions considerades:

`Array (int[])` per a les assignacions de clúster

Avantatges: accés per índex en temps constant $\Theta(1)$, simplicitat i eficiència en memòria.

Inconvenients: mida fixa, no permet afegir ni eliminar usuaris dinàmicament.

Decisió: escollit, ja que cada usuari té exactament un clúster assignat i no cal flexibilitat.

`ArrayList<ArrayList<AnswerToQuestion>>` per als centroides

Avantatges: flexibilitat per representar una matriu dinàmica, permet afegir o eliminar centroides i gestionar les respostes de manera senzilla.

Inconvenients: insercions i eliminacions al mig de la llista tenen cost $\Theta(n)$.

Decisió: escollit, ja que ofereix una representació clara i flexible dels centroides, mantenint l'ordre i permetent operacions dinàmiques sobre les respostes.

Altres opcions

`LinkedList`: descartada perquè l'accés per índex és lent $\Theta(n)$ i en aquest cas es fan moltes consultes directes.

`HashMap`: podria representar centroides amb claus, però es perdria l'ordre natural i la simplicitat de la representació matricial.

`AnswerToQuestion[][]`: hauria estat més eficient en memòria i accés, però menys flexible per gestionar dinàmicament els centroides.

Finalment, es va escollir una combinació de `int[]` per a les assignacions de clúster i `ArrayList<ArrayList<AnswerToQuestion>>` per a la representació dels centroides. Aquesta decisió es basa en la simplicitat i eficiència de l'array per a dades fixes, i en la flexibilitat de l'`ArrayList` per gestionar col·leccions dinàmiques de respostes.

4.2.5 Analyze

Tal i com hem especificat anteriorment, la classe `Analyze` guarda diversos elements entre ells la següent estructura:

`ArrayList<ArrayList<AnswerToQuestion>>` answers: matriu dinàmica que conté totes les respostes dels usuaris al formulari, organitzades per files (respostes completes) i columnes (preguntes individuals).

Opcions considerades:

Array bidimensional (`AnswerToQuestion[][]`)

Avantatges: accés per índex en temps constant $\Theta(1)$, eficiència en memòria.

Inconvenients: mida fixa, poca flexibilitat per afegir o eliminar respostes.

No escollit: en un sistema on les respostes poden variar, la rigidesa d'un array no és adequada.

`ArrayList<ArrayList<AnswerToQuestion>>`

Avantatges: estructura dinàmica, permet afegir i eliminar respostes fàcilment, manté l'ordre de les respostes i les preguntes.

Inconvenients: insercions i eliminacions al mig de la llista tenen cost $\Theta(n)$.

Escollit: és la millor opció per representar una matriu de respostes que pot créixer o reduir-se dinàmicament.

Altres opcions

LinkedList: descartada perquè l'accés per índex és lent $\Theta(n)$, i en aquest cas es fan moltes consultes directes.

HashMap: podria representar respostes amb claus (per exemple, `idUsuari` → respostes), però es perdria l'ordre natural i la simplicitat de la representació matricial.

Finalment, es va escollir `ArrayList<ArrayList<AnswerToQuestion>>` per representar les respostes dins la classe `Analyze`. L'ús d'`ArrayList` garanteix un bon equilibri entre flexibilitat i eficiència, aspectes clau en el context d'una anàlisi de clustering aplicada a formularis.

4.2.6 AnalyzeStrategy

Tal i com hem especificat anteriorment, la classe `AnalyzeStrategy` guarda diversos elements entre ells la següent estructura: `ArrayList<ArrayList<AnswerToQuestion>>` answers: matriu dinàmica que conté totes les respostes dels usuaris al formulari, organitzades per files (respostes completes) i columnes (preguntes individuals). A més, en els mètodes de càlcul de distàncies locals també s'utilitzen col·leccions de tipus `ArrayList<Integer>` per representar les opcions seleccionades en preguntes de resposta múltiple.

Opcions considerades:

Array bidimensional (`AnswerToQuestion[][]`)

Avantatges: accés per índex en temps constant $\Theta(1)$, eficiència en memòria.

Inconvenients: mida fixa, poca flexibilitat per afegir o eliminar respostes.

No escollit: en un sistema on les respostes poden variar, la rigidesa d'un array no és adequada.

`ArrayList<ArrayList<AnswerToQuestion>>`

Avantatges: estructura dinàmica, permet afegir i eliminar respostes fàcilment, manté l'ordre de les respostes i les preguntes.

Inconvenients: insercions i eliminacions al mig de la llista tenen cost $\Theta(n)$.

Escollit: és la millor opció per representar una matriu de respostes que pot créixer o reduir-se dinàmicament.

`ArrayList<Integer>` (per opcions seleccionades)

Avantatges: simplicitat, accés per índex ràpid $\Theta(1)$, mètodes ja implementats (add, remove, contains).

Inconvenients: insercions/eliminacions al mig de la llista tenen cost $\Theta(n)$.

Escollit: suficient per representar conjunts d'opcions seleccionades en preguntes de tipus múltiple.

Altres opcions

`LinkedList`: descartada perquè l'accés per índex és lent $\Theta(n)$, i en aquest cas es fan moltes consultes directes.

`HashSet`: podria representar respostes o opcions seleccionades evitant duplicats i amb operacions ràpides $\Theta(1)$, però es perdria l'ordre natural i la simplicitat de la representació matricial.

Finalment, es va escollir `ArrayList<ArrayList<AnswerToQuestion>>` per representar les respostes dins la classe `AnalyzeStrategy` i `ArrayList<Integer>` per gestionar les opcions seleccionades en preguntes de resposta múltiple.

4.2.7 KMeansStrategy

Per a executar l'algorisme k-means hem utilitzat les següents estructures de dades:

`ArrayList<ArrayList<AnswerToQuestion>>` answers: matriu que conté totes les respostes dels usuaris al formulari, organitzades per files (respostes completes) i columnes (preguntes individuals).

Hem triat aquesta representació perquè guarda la informació necessària per al còmput de les distàncies i és intuïtiu i més senzill de crear que les alternatives.

Alternatives:

`Object[][]` -> seguiria sent el mateix sistema però en comptes de guardar-nos l'instància d'`AnswerToQuestion` ens guardaríem sols la resposta donada per l'usuari (i com que tenim diversos tipus de valors possibles necessitem que sigui una matriu de `Object`).

En aquest cas la primera fila contindria un `String` per cada columna on posaria de quin tipus és la pregunta de la columna (perquè les numèriques i les qualitatives ordenades es representen ambdues amb un enter i per tant no podríem distingir-les) i informació adicional (en el cas de les numèriques el valor màxim i el mínim, en el de les qualitatives ordenades, el nombre de modes possibles).

Aquesta alternativa representa una millora substancial en termes d'espai (perquè no ens guardem la classe sencera) i sobretot d'acoblament, car voldria dir que l'algorisme no depèn de cap altra classe del diagrama.

El motiu pel qual no l'hem triada és perquè era poc intuïtiu treballar amb ella i perquè era més costosa de generar. I és que per a crear-la havíem de crear un `ArrayList<ArrayList<AnswerToQuestion>>` de totes formes.

Tot i així és una candidata sòlida

`ArrayList<Answer>` -> Aquesta alternativa empitjora l'alternativa triada perquè tot i que sigui més intuïtiu perquè treballem directament amb les respostes ens dificulta la creació de centroides i afegia acoblament fàcilment evitable.

`ArrayList<Integer>` centroids: llista dinàmica que guarda els índexs de les respostes seleccionades com a centroides. Treballem amb índexs perquè és més eficient en espai. No utilitzem `Integer[]` perquè necessitem afegir dinàmicament.

`ArrayList<Double>` probabilities: llista que representa la distribució de probabilitats utilitzada en la inicialització k-means++. En aquest cas és equivalent a utilitzar un `Double[]`.

4.2.8 KMedoidsStrategy

La classe *KMedoidsStrategy* implementa l'algorisme de k-medoids amb inicialització aleatòria de medoides. Aquesta classe fa ús de diverses estructures de dades per gestionar tant els centroides (medoides) com les assignacions de clúster i les distàncies precalculades entre usuaris. Entre elles destaquen:

`int[] medoidIndices`: array que guarda els índexs dels usuaris seleccionats com a medoids.

`int[] clusterAssignments`: array que indica a quin cluster pertany cada usuari.

`double[][] distances`: matriu simètrica que emmagatzema les distàncies precalculades entre cada parella d'usuaris.

`ArrayList<ArrayList<AnswerToQuestion>>` answers (paràmetre dels mètodes): matriu dinàmica que conté totes les respostes dels usuaris al formulari, organitzades per files (respostes completes) i columnes (preguntes individuals).

Opcions considerades:

Array bidimensional (`AnswerToQuestion[][]`)

Avantatges: accés per índex en temps constant $\Theta(1)$, eficiència en memòria.

Inconvenients: mida fixa, poca flexibilitat per afegir o eliminar respostes.

`ArrayList<ArrayList<AnswerToQuestion>>`

Avantatges: estructura dinàmica, permet afegir i eliminar respostes fàcilment, manté l'ordre de les respostes i les preguntes.

Inconvenients: insercions i eliminacions al mig de la llista tenen cost $\Theta(n)$.

Escollit: és la millor opció per conveniència, realment no modificarem la mida d'aquesta matriu dins la classe de l'algorisme.

4.2.9 Persistència

Per a poder accedir des de la capa de Domini a funcions de la capa de persistència s'ha implementat una classe *CtrlPersistence*, que defineix certes operacions bàsiques.

En aquesta classe s'han definit funcions `addX(X x)`, `getX(id...)`, `deleteX(id...)` i `getAllX()` per a les classes del domini *User*, *Form*, *Question*, *Answer*, *AnswerToQuestion* i *Analyze*.

Aquest CtrlPersistence s'ha subdividit en quatre controladors més petits i específics: ctrlUser (que tracta User), ctrlFormQuestion (que tracta Form i Question), ctrlAnalyze (que tracta Analyze) i ctrlAnswerQuestion (que tracta Answer i AnswerToQuestion).

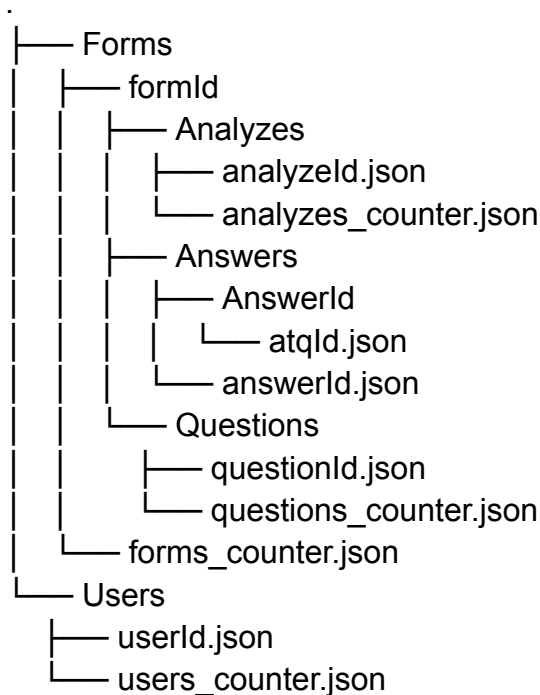
Cada controlador té accés a la classe genèrica Cache, que és l'encarregada de guardar les dades en memòria (usant un Map<K, V>, on K key és l'identificador de l'objecte i V value és l'objecte a guardar).

Aquesta cache s'implementa com a WriteAllocate i una variant de CopyBack (la diferència que sempre que un objecte és reemplaçat l'escriu a disc i no segueix si s'ha modificat o no).

Per a guardar les dades a disc, de manera persistent, s'usa una classe FileManager, encarregada d'usar els Reader i Writer que el llenguatge Java proporciona.

Aquesta classe, el que fa és escriure a disc fitxers Json. En el nostre cas un fitxer json correspon a una instància d'una classe, i usem la llibreria Gson per a convertir de instància de classe a json i viceversa.

Pel que fa a l'estructura de dades al disc, ens hem decidit a implementar-ho de la manera següent:



Repartint la informació en molts arxius diferents ens assegurem que els arxius a guardar i carregar són petits, és a dir, aquestes operacions es faran de manera ràpida; sense afegir temps de latència per als usuaris. A més, permet mantenir la informació molt explícita i ben estructurada en disc.

5 Milliores en algorismes i estructures de dades

5.1 Algorismes

5.1.1 K-Medoids

De cara a la segona entrega s'ha fet una millora del K-Medoids, passant d'una implementació PAM amb pre-càlcul de distàncies a un Fast PAM amb pre-càlcul de distàncies.

A diferència de la implementació anterior, en aquesta, quan s'avaluen els possibles canvis de medoid amb no medoid, no simulam el swap i calculem la distància total en cada iteració, sinó que ho fem de manera incremental, mirant en cada cas, quina és la distància que sumam per desassignar cada punt amb el medoid que "eliminam" i restam la distància de tots els punts al nou medoid que "cream". Això ho podem fer perquè abans d'executar la

funció que evalua els swaps, guardam per a cada punt (respostes usuari) la distància al medoid més proper i la distància al segon medoid més proper amb temps $O(n \cdot k)$. Aquesta millora ens permet reduir la complexitat temporal en k , sent k el nombre de medoids. El cost final quedaria de la següent manera: $O(n^2 \cdot k)$. Segueix sent una implementació determinista, sempre obtindrem el mateix resultat per a una mateixa entrada.

5.2 Estructures de dades

Veure l'apartat 4.2.9 de Persistència.